



山东大学

信息科学与工程学院

2024 – 2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: Huffman 编码的编程与实现

一、实验目的

掌握 Huffman 树的概念、存储结构；掌握建立 Huffman 树和 Huffman 编码的方法

二、实验原理

参照课本 p. 266-270

Huffman 编码是一种常用的变长编码方法，其基本思想是：根据数据出现的频率或权值构造一棵 Huffman 树，使权值较大的结点距离根结点较近，权值较小的结点距离根结点较远，从而使整体带权路径长度最小。通过这种方式，可以让出现频率高的数据使用较短编码，出现频率低的数据使用较长编码，从而达到压缩存储空间的目的。

本实验的主要内容是利用动态分配数组存储 Huffman 树，并根据输入的数据构造 Huffman 树，最后输出对应的 Huffman 编码。实验要求包括创建动态顺序表、构造 Huffman 树、根据 Huffman 树生成编码并输出编码结果。

Huffman 树的构造过程如下：首先将每个输入字符或数据看作一个只有根结点的二叉树，每个结点都有一个对应的权值。然后在所有尚未合并的结点中选择权值最小的两个结点，分别作为新结点的左孩子和右孩子，并将它们的权值之和作为新结点的权值。重复执行这一过程，直到所有结点合并成一棵二叉树为止，这棵树就是 Huffman 树。

在生成 Huffman 编码时，从每个叶子结点开始向根结点回溯。如果当前结点是其双亲结点的左孩子，则记为 0；如果是右孩子，则记为 1。由于编码是从叶子结点向根结点逆向生成的，所以最后需要将得到的编码反向排列，才能得到该字符真正的 Huffman 编码。

Huffman 编码具有前缀编码的特点，即任意一个字符的编码都不是另一个字符编码的前缀，因此在译码时不会产生歧义。通过本实验，可以进一步理解 Huffman 树的顺序存储结构、构造方法以及编码生成过程

三、实验内容

利用动态分配数组存储赫夫曼树，设计一组输入数据（假定为一组整数），能够对其进行如下操作：

- 1) 创建一个新的顺序表，实现动态空间分配的初始化
- 2) 对输入的数据构造成一棵 Huffman 树
- 3) 根据生成的 Huffman 树进行 Huffman 编码；
- 4) 实现对输入数据的 Huffman 编码输出

四、实验要求

- 1) 实现 Huffman 树的生成
- 2) 完成 Huffman 编码的输出

五、实验源程序

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>

#define MAX_NODES 20
#define MAX_VALUE 1000

typedef struct {
    char ch;
    int weight;
    int lChild;
    int rChild;
    int parent;
    char hCode[MAX_NODES];
} HuffmanNode;

void initializeHuffmanTree(HuffmanNode* tree, int* weights, char* chars,
int n) {
    for (int i = 0; i < 2 * n - 1; i++) {
        tree[i].ch = i < n ? chars[i] : '\0';
        tree[i].weight = i < n ? weights[i] : 0;
        tree[i].parent = -1;
        tree[i].lChild = -1;
        tree[i].rChild = -1;
        memset(tree[i].hCode, 0, sizeof(tree[i].hCode));
    }
}

void buildHuffmanTree(HuffmanNode* tree, int n) {
    for (int i = n; i < 2 * n - 1; i++) {
        int min1 = MAX_VALUE, min2 = MAX_VALUE;
        int idx1 = -1, idx2 = -1;

        for (int j = 0; j < i; j++) {
            if (tree[j].parent == -1) {
                if (tree[j].weight < min1) {
                    min2 = min1;
                    idx2 = idx1;
                    min1 = tree[j].weight;
                    idx1 = j;
                } else if (tree[j].weight < min2) {
                    min2 = tree[j].weight;
                    idx2 = j;
                }
            }
        }
    }
}
    
```

```

        if (idx1 != -1 && idx2 != -1) {
            tree[idx1].parent = i;
            tree[idx2].parent = i;
            tree[i].lChild = idx1;
            tree[i].rChild = idx2;
            tree[i].weight = min1 + min2;
        }
    }
}

void generateHuffmanCodes(HuffmanNode* tree, int n) {
    char tempCode[MAX_NODES];
    for (int i = 0; i < n; i++) {
        int current = i;
        int parent = tree[current].parent;
        int codeLen = 0;

        while (parent != -1) {
            if (tree[parent].lChild == current) {
                tempCode[codeLen++] = '0';
            } else {
                tempCode[codeLen++] = '1';
            }
            current = parent;
            parent = tree[current].parent;
        }

        // Reverse the code
        for (int j = 0; j < codeLen; j++) {
            tree[i].hCode[j] = tempCode[codeLen - 1 - j];
        }
        tree[i].hCode[codeLen] = '\0';
    }
}

int main() {
    int nodeCount;
    printf("请输入叶子节点数量: \n");
    scanf("%d", &nodeCount);

    if (nodeCount <= 0 || nodeCount > MAX_NODES) {
        printf("无效的节点数量! \n");
        return 1;
    }
}

```

```

}

char characters[MAX_NODES];
int weights[MAX_NODES];
HuffmanNode huffmanTree[2 * MAX_NODES - 1];

printf("请输入%d 个字符及其权值: \n", nodeCount);
for (int i = 0; i < nodeCount; i++) {
    printf("请输入第%d 个字符: \n", i + 1);
    scanf(" %c", &characters[i]);
    printf("请输入第%d 个字符的权值: \n", i + 1);
    scanf("%d", &weights[i]);
}

initializeHuffmanTree(huffmanTree, weights, characters, nodeCount);
buildHuffmanTree(huffmanTree, nodeCount);
generateHuffmanCodes(huffmanTree, nodeCount);

printf("\n 生成的哈夫曼编码: \n");
for (int i = 0; i < nodeCount; i++) {
    printf("%c: %s\n", huffmanTree[i].ch, huffmanTree[i].hCode);
}

return 0;
}

```

六、实验结果

```

请输入叶子节点数量：
3
请输入3个字符及其权值：
请输入第1个字符：
W
请输入第1个字符的权值：
2
请输入第2个字符：
L
请输入第2个字符的权值：
10
请输入第3个字符：
C
请输入第3个字符的权值：
7

生成的哈夫曼编码：
W: 00
L: 1
C: 01
    
```

七、实验心得

在本次实验中，我完成了 Huffman 树的构造和 Huffman 编码的输出，对 Huffman 编码的基本原理和实现过程有了更加直观的理解。本实验要求利用动态分配数组存储 Huffman 树，并根据输入数据生成 Huffman 树和对应编码。通过程序实现，我进一步掌握了 Huffman 树的顺序存储结构，以及如何通过结点的权值、双亲、左孩子和右孩子等信息表示一棵二叉树。

在实验过程中，我重点理解了 Huffman 树的构造方法。构造时每次都要从当前没有双亲的结点中选择两个权值最小的结点，将它们合并成一个新的结点，新结点的权值等于两个子结点权值之和。这个过程体现了贪心算法的思想，即每一步都选择当前最优的两个结点进行合并，最终得到带权路径长度较小的 Huffman 树。

在生成 Huffman 编码时，我认识到编码并不是从根结点直接存储得到的，而是可以从叶子结点向根结点回溯生成。如果当前结点是双亲结点的左孩子，则记为 0；如果是右孩子，则记为 1。由于回溯得到的编码顺序是反的，所以在输出前需要进行反向处理。通过这一过程，我加深了对 Huffman 编码“左 0 右 1”和前缀编码特点的理解。

通过本次实验，我不仅巩固了 Huffman 树和 Huffman 编码的理论知识，也提高了自己使用结构体、动态内存分配和数组顺序存储结构解决实际问题的能力。在调试过程中，我对结点下标、双亲关系和编码生成顺序有了更清楚的认识。总体来说，本次实验帮助我把课本中的 Huffman 编码原理转化为具体程序实现，为后续学习数据压缩和树结构相关算法打下了基础。